

ELECTRONICS GUIDE | جزوهٔ آموزشی مهندسی الکترونیک

راهنمای جامع
دستورهای اختصاصی هسته:
XW، MCPY و DLY

از انگیزهٔ طراحی، تعریف کارکرد و تعریف فیلد بیت‌ها، تا سنجش کمّی بهرهٔ کارایی در الگوریتم‌های خاص و هزینه‌های سخت‌افزاری دستورهای توسعه‌یافتهٔ هسته‌های RISC-V

در این جزوه، ترجمهٔ فارسی سند رسمی AN03003 (نسخهٔ ۱٫۰، منتشرهٔ شرکت WCH) را می‌خوانید: دستورهای اختصاصی هسته در تراشه‌های خودساختهٔ شرکت بر پایهٔ معماری RISC-V. ان توسعهٔ فشردهٔ دستورهای بایت و نیم‌کلمه (XW)، دستور سخت‌افزاری کپی حافظه (MCPY) با جایگزینی memcpy و پایش آزمون عملکرد، و دستور تأخیر دقیق (DLY) با کنترل خط لوله و رجیسترهای CSR همراه تعریف فیلد بیت‌ها، نحوهٔ مونتاژ و نکات استفاده بررسی می‌شود.

توسعهٔ دستورهای فشردهٔ XW | کپی سخت‌افزاری MCPY | تأخیر دقیق DLY | تعریف فیلد بیت‌ها
رجیسترهای CSR | کنترل خط لوله | ابزار زنجیرهٔ WCH | واژه‌نامهٔ فنی

ترجمهٔ فارسی AN03003 V1.0 · نسخهٔ ۱٫۰ — آبان ۱۴۰۵

فهرست مطالب — برای پرش به بخش موردنظر، روی ردیف آن کلیک کنید؛ شمارهٔ صفحه‌ها بر اساس صفحات فیزیکی PDF است.

فهرست مطالب

۱ توضیحات و دامنهٔ کاربرد ۳

۲ بخش ۱ | دستورهای فشردهٔ توسعه‌یافته XW ۳

۳ بخش ۲ | انتقال سخت‌افزاری MCPY ۵

2.1 معرفی ۵

2.2 آزمون عملکرد ۶

2.3 استفاده و نکات ۷

۴ بخش ۳ | تأخیر دقیق DLY ۸

3.1 معرفی ۸

3.2 ویژگی‌ها ۹

3.3 استفاده ۱۰

۵ تاریخچهٔ نسخه‌ها و بیانیه ۱۲

۶ واژگان کلیدی (واژه‌نامهٔ فنی) ۱۲

فهرست شکل‌ها، جدول‌ها و کدها

جدول ۱-۱ مرور دستورهای فشردهٔ توسعهٔ XW ۴

جدول ۲-۱ تعریف فیلد بیت‌های دستور MCPY ۵

شکل ۲-۱ مقایسه با انتقال سنتی memcpy ۷

جدول ۳-۱ تعریف فیلد بیت‌های دستور تأخیر دقیق DLY ۸

جدول ۳-۲ رجیسترهای CSR همراهِ DLY ۹

جدول ۳-۳ رابط‌های قابل فراخوانی در کتابخانهٔ رسمی فعلی ۱۰

کد ۱ کد آزمون سنجش عملکرد با ucycle ۶

کد ۲ نمونهٔ ساده استفاده از DLY ۱۱

چکیده — این سند به بیان منظّم مجموعه دستورهای ویژهٔ هسته‌ای می‌پردازد که شرکت در تراشه‌های خودساختهٔ مبتنی بر معماری RISC-V تعبیه کرده است: انگیزهٔ طراحی ریزمعماری، تعریف کارکرد و تحلیل کمّی بهرهٔ کارایی در سناریوهای الگوریتمی خاص و هزینه‌های سخت‌افزاری، برای ارزش‌گذاری کامل از ارزش هسته‌ای این دستورها در بهبود نسبت کارایی به توان پردازنده.

مقدمه

توضیحات و دامنهٔ کاربرد

این سند به بیان منظّم مجموعه‌ای از دستورهای ویژهٔ هسته می‌پردازد که شرکت در تراشه‌های خودساختهٔ مبتنی بر معماری RISC-V تعبیه کرده است. محتوا انگیزهٔ طراحی ریزمعماری و تعریف کارکرد این دستورها را در بر می‌گیرد و بهرهٔ کارایی در سناریوهای الگوریتمی خاص و هزینه‌های سخت‌افزاری را به‌صورت کمّی تحلیل می‌کند تا ارزش هسته‌ای آنها در ارتقای نسبت کارایی به توانِ پردازنده به‌طور کامل ارزیابی شود.

جدول ۱ دامنهٔ کاربرد سند

دامنهٔ کاربرد	سری
میکروکنترلرهای همگانی‌منظوره	CH32

بخش ۱

دستورهای فشردهٔ توسعه‌یافته (XW)

◆ ۱.۱ تعریف فیلد بیت‌ها

توسعهٔ دستورهای فشردهٔ استاندارد RISC-V (توسعهٔ C) عمدتاً عملیات بارگذاری و ذخیرهٔ کلمه و کلمهٔ مضاعف (RV64C) را فشرده کرده است؛ اما در توسعهٔ واقعیِ نهفته، عملیات بایت (Byte) و نیم‌کلمه (Half-Word) نیز بسیار پرتکرارند. توسعهٔ XW دقیقاً برای پرکردن همین خلأ پدید آمده است: ۸ دستور فشردهٔ ۱۶ بیتی جدید که ویژهٔ بهینه‌سازی عملیات بایت و نیم‌کلمه‌اند و حجم نهایی فایل دودویی را به‌طور مؤثر کم می‌کنند؛ برای MCUهایی که فلش و SRAM آنها معمولاً بزرگ نیست، این موضوع معنای ویژه‌ای دارد.

جدول ۱-۱ مرور دستورهای فشردهٔ توسعهٔ XW (چیدمان بیت‌های ۱۵ تا ۰)

استاندارد RISC-V	XW	بیت‌های [15:0]
c.fld / c.lq	c.lbu	001 uimm[0|4:3] rs1' uimm[2:1] rd' 00
c.fldsp / c.lqsp	c.lhu	001 uimm[5|4:3] rs1' uimm[2:1] rd' 10
c.fsd / c.sq	c.sb	101 uimm[0|4:3] rs1' uimm[2:1] rs2' 00
c.fsdsp / c.sqsp	c.sh	101 uimm[5|4:3] rs1' uimm[2:1] rs2' 10
c.lw	c.lw	010 uimm[5:3] rs1' uimm[2|6] rd' 00
c.sw	c.sw	110 uimm[5:3] rs1' uimm[2|6] rs2' 00
Reserved	c.lbusp	100 00 uimm[3:0] 00 rd' 00
Reserved	c.lhusp	100 00 uimm[3:1|4] 01 rd' 00
Reserved	c.sbsp	100 00 uimm[3:0] 10 rs2' 00
Reserved	c.shsp	100 00 uimm[3:1|4] 11 rs2' 00
c.lwsp	c.lwsp	010 uimm[5] rd != 0 uimm[4:2|7:6] 10
c.swsp	c.swsp	110 uimm[5:2|7:6] rs2 10

هسته‌های این سری، ۸ دستور فشردهٔ خواندن/نوشتن سفارشی را پیاده‌سازی کرده‌اند؛ کارکردشان با مراجعه به جفت‌دستورهای c.lw/c.sw/c.lwsp/c.swsp، عملیات دادهٔ نیم‌کلمه و بایت را تکمیل می‌کند. دستورهای c.lbu/c.lhu/c.sb/c.sh فضای کدگذاری دستورهای فشردهٔ ممیز شناور دودویی را اشغال کرده‌اند و وضعیت تعارض مطابق جدول بالا است.

شرایط استفاده از دستورهای فشردهٔ توسعهٔ XW: توسعهٔ XW مجموعه دستوری سفارشیِ WCH بر پایهٔ معماری استاندارد RISC-V است و دستورهای فشردهٔ آن فقط در محیط زنجیرهٔ ابزار مشخصی کامپایل و اجرا می‌شوند. الزامات: **۱) الزام زنجیرهٔ ابزار** — باید از زنجیرهٔ ابزار رسمی RISC-V شرکت WCH استفاده شود؛ زنجیرهٔ ابزار متنباز استاندارد RISC-V (مثل نسخهٔ رسمی GNU) به‌طور پیش‌فرض پشتیبانی توسعهٔ XW را ندارد و دستورهای مرتبط را تشخیص نمی‌دهد یا تولید نمی‌کند. **۲) الزام پارامتر کامپایل** — هنگام کامپایل باید در پارامتر march- شناسهٔ توسعهٔ xw صراحتاً اضافه شود تا کامپایلر مطلع شود پشتیبانی توسعهٔ XW را فعال کند.

بخش ۲

انتقال سخت‌افزاری MCPY

◆ ۲.۱ معرفی

این دستور کپی پیوستهٔ بایت‌های حافظه از آدرس مبدأ به آدرس مقصد را سخت‌افزاری انجام می‌دهد: rs1 آدرس پایان کپی (خود آن آدرس شامل نمی‌شود)، rs2 آدرس آغاز کپی و rd آدرس آغاز مقصد است. در طول اجرای دستور، سخت‌افزار به‌طور خودکار از آدرس آغاز مبدأ بایت‌به‌بایت می‌خواند و در آدرس آغاز مقصد می‌نویسد، تا وقتی آدرس مبدأ به آدرس پایان برسد. این دستور جانشین تابع memcpy است و انتقال پیوستهٔ حافظه را انجام می‌دهد؛ رجیسترهای دستور اطلاعات آدرسِ آغاز، مقصد و پایان را نگه می‌دارند و هیچ‌یک از آدرس‌ها الزام هم‌ترازی ندارند.

◆ ۲.۱.۱ اطلاعات دستور

تعریف فیلد بیت‌ها (Format):

جدول ۲-۱ تعریف فیلد بیت‌های دستور MCPY

نام	بیت	کارکرد
rs3	31:27	کد رجیستر آدرس پایان
RSV	26:25	مقدار ثابت 00
rs2	24:20	کد رجیستر آدرس آغاز
rs1	19:15	کد رجیستر آدرس مقصد
fun3	14:12	مقدار ثابت 111
fun5	11:7	مقدار ثابت 00000
fun7	6:0	مقدار ثابت 0001111

نحوهٔ مونتاژ (Syntax):

```
mcpy rs1, rs2, rs3
```

منطق اجرا (Pseudocode):

```
while (rs2 < rs1) {
mem[rs3] = mem[rs2];
rs2 = rs2 + 1;
rs3 = rs3 + 1;
}
```

پاسخ وقفه: این دستور قابل وقفه است. در طول اجرای mcpy اگر درخواست وقفهٔ معتبر شناسایی شود، سخت‌افزار پس از اتمام انتقال بایت جاری (یا در مرز دور دستور بعدی) اجرا را موقتاً متوقف می‌کند. پس از بازگشت وقفه، دستور ادامه می‌یابد و روند کپی با حالتِ بدون وقفه، نتیجهٔ نهایی یکسان دارد. این سازوکار توان پاسخ‌گویی لحظه‌ای سیستم را در سناریوهای انتقال بلوک دادهٔ بزرگ تضمین می‌کند.

◆ ۲.۲ آزمون عملکرد

روش ارزیابی عملکرد: برای کمّی‌کردن کارایی اجرای شیوه‌های مختلف انتقال داده در حالت غیرممتاز، این آزمون از رجیستر ucycle هستهٔ RISC-V (آدرس 0xC00) برای شمارش دقیق دورها استفاده می‌کند. سطح بهینه‌سازی -OFast است تا بیشترین کارایی کتابخانه حاصل شود.

روند آزمون به این شکل است:

۱ با دستور مونتاژ درون‌خطی csrr تعداد دور ماشین جاری خوانده و به‌عنوان نشانِ زمان آغاز (starttime) ثبت می‌شود.

۲ تابع انتقال دادهٔ تحت آزمون CopyMethod(Adapter, mydata, range) فراخوانی می‌شود.

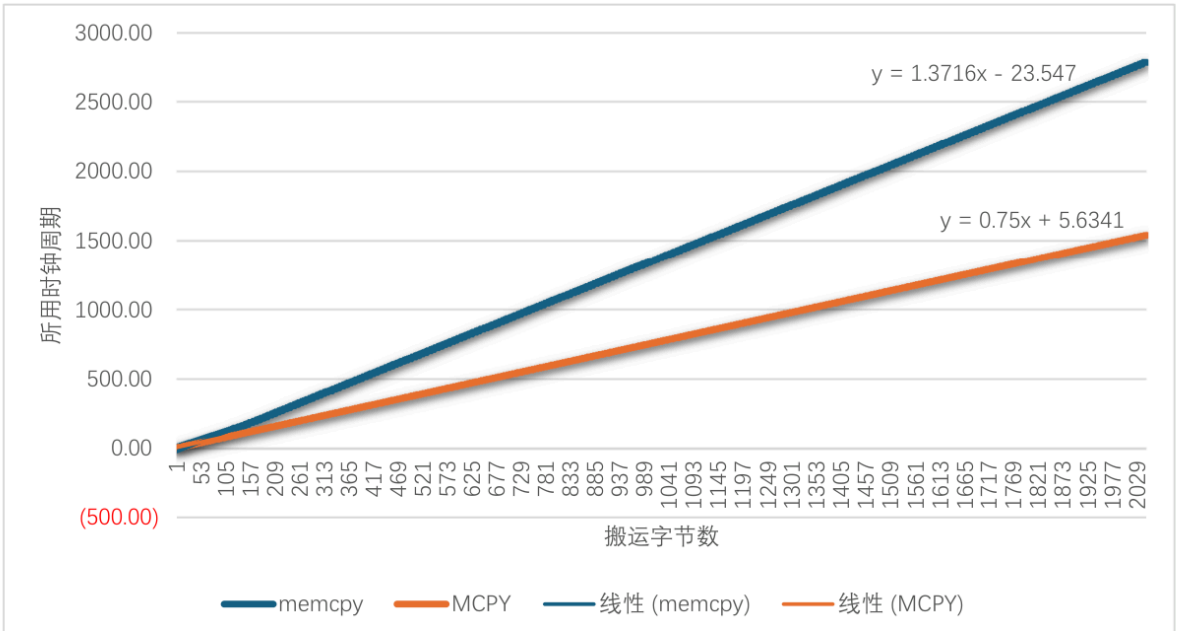
۳ پس از پایان انتقال، رجیستر ucycle دوباره خوانده می‌شود و نشانِ زمان پایان (stoptime) به دست می‌آید.

کد آزمون به این شکل است:

```
__asm volatile("csrr %0,"
"0xC00"
: "=r"(starttime)
:);
CopyMethod(Adapter, mydata, range);
__asm volatile("csrr %0,"
"0xC00"
: "=r"(stoptime)
:);
```

کد ۱ شمارش دورهای مصرفی با رجیستر ucycle

در این آزمون در بازهٔ range = 1~2048 بایت، مصرف دور ساعت دو شیوهٔ انتقال داده نقطه‌به‌نقطه آماری شده و بر این اساس منحنی عملکرد رسم شده است (شکل زیر). نتایج آزمایش نشان می‌دهد در کل بازهٔ آزمون، هزینهٔ دور ساعتِ هر دو شیوهٔ انتقال با حجم دادهٔ منتقل‌شده رابطهٔ خطی تقریباً مثبت دارد.



شکل ۲-۱ مقایسه با انتقال سنتی memcpy

با برازش خطی روی داده‌های اندازه‌گیری‌شده، عبارت‌های برازش دو روش عبارت‌اند از: y = 1.3716x - 23.547 (برای memcpy) و y = 0.75x + 5.6341 (برای MCPY). از نتیجهٔ برازش و مقایسهٔ شیب منحنی‌ها روشن است که روش MCPY در کارایی انتقال تک‌بایت به‌طور چشمگیر بر روش متعارف memcpy برتری دارد؛ به‌ویژه وقتی حجم دادهٔ منتقل‌شده بزرگ‌تر می‌شود، صرفه‌جویی تجمعی دورهای MCPY چشمگیرتر است و زمان اشغال CPU در وظایف انتقال داده را به‌طور مؤثر کم می‌کند و توان عملیاتی کل سیستم را بالا می‌برد.

◆ ۲.۳ استفاده و نکات

شیوهٔ فراخوانی: نحوهٔ مونتاژ دستور MCPY را بخش ۱ همین فصل شرح داده است. در محیط کتابخانهٔ رسمی توسعهٔ نرم‌افزار (SDK) کاربر می‌تواند مستقیماً رابط کپسول‌شدهٔ ASM_MCPY() را برای استفاده از این دستور فراخوانی کند.

نکات:

۱ **محدودیت هم‌ترازی و هم‌پوشانی آدرس‌ها:** دستور MCPY انتقال داده را به‌صورت ترتیبی از آدرس پایین به آدرس بالا انجام می‌دهد. استفاده از این دستور وقتی انتهای ناحیهٔ دادهٔ مبدأ و آغاز ناحیهٔ دادهٔ مقصد هم‌پوشانی دارند به‌سختی ممنوع است؛ در چنین هم‌پوشانی، دادهٔ مبدأ پیش از خوانده‌شدن، توسط عمل نوشتن مقصد بازنویسی می‌شود و نتیجهٔ انتقال به خطا می‌رود.

۲ **رفتار پاسخ وقفه:** در طول اجرای دستور MCPY پاسخ به وقفه مجاز است. پس از پایان اجرای تابع سرویس وقفه، دستور به‌طور خودکار ادامه می‌یابد و نیازی به اقدام اضافی نرم‌افزار نیست. درخواست دیباگ (شامل تک‌قدمی) پس از اتمام کامل اجرای دستور جاری MCPY پاسخ می‌گیرد و روند انتقال را قطع نمی‌کند.

۳ **مدیریت استثنا:** اگر آدرس مبدأ یا مقصد دستور MCPY سازوکارهای حفاظتی زیر را فعال کند، استثنای غیرقابل بازیابی رخ می‌دهد و سیستم باید ریست سطح بالا یا مدیریت خطا انجام دهد: محدودیت مجوز دسترسی حفاظت حافظهٔ فیزیکی (PMP)؛ برخورد با نقطهٔ شکست سخت‌افزاری (Hardware Breakpoint).

۴ **اثر جانبی رجیسترها:** در طول اجرای دستور MCPY، رجیستر آدرس مبدأ (rs2) و رجیستر آدرس مقصد (rs3) همراه با پیشرفت انتقال به‌طور افزایشی به‌روز می‌شوند. پس از پایان دستور، مقدار نهایی rs2 و rs3 به موقعیت آدرس در لحظهٔ پایانِ انتقال وابسته و غیرقابل پیش‌بینی است؛ کاربر نباید در برنامه‌نویسی به مقدارهای بعدی آنها تکیه کند.

بخش ۳

تأخیر دقیق DLY

◆ ۳.۱ معرفی

دستور تأخیر سفارشی، یک دستور تأخیر با دقت بالا است که بهای آن فقط هزینهٔ یک دستور است. این دستور کنترل تأخیر را تا سطح تک‌دور ساعت دقیق می‌کند و به‌طور مؤثر از مشکلات عدم‌قطعیت زمان‌بندیِ روش‌های سنتی تأخیر (تایمر یا حلقهٔ نرم‌افزاری) ناشی از پرش دستور، خالی‌شدن خط لوله و شکست پیش‌بینی انشعاب جلوگیری می‌کند.

◆ ۳.۱.۱ اطلاعات دستور

جدول ۳-۱ تعریف فیلد بیت‌های دستور تأخیر دقیق DLY

نام	بیت	کارکرد
imm	31:20	عدد تأخیر؛ بسته به پیکربندی، تعداد دورهای ساعت تأخیر یا تعداد دورهای پس از تقسیم فرکانس را نشان می‌دهد.
rs1	19:15	کد رجیسترِ rs1 متناظر برای تطبیق.
func3	14:12	.001b
match	11:9	نوع دستورِ تطبیق‌شده؛ تا پایان تأخیر، نوع دستورِ تطبیق‌شده اجرا نمی‌شود: 000: محفوظ؛ 001: تطبیق دستور load؛ 010: تطبیق دستور store؛ 011: تطبیق دستور load/store؛ 100: تطبیق دستور delay؛ 101: محفوظ؛ 110: تطبیق همهٔ دستورها (تعلیق خط لوله)؛ 111: تطبیق دستور با func3 و opcode مشخص (مقدار تطبیق با رجیستر csr پیکربندی می‌شود).
div	8	شمارش با ساعت اصلی یا ساعت تقسیم‌شده. 1: شمارش با دور پس از تقسیم (ضریب تقسیم با رجیستر csr پیکربندی می‌شود)؛ 0: شمارش با دورِ ساعت اصلی.
sel	7	انتخاب مقدار تأخیر: 1: rs2 + rs1؛ 0: imm + rs1.
fun7	6:0	0001011b

۳.۱.۲ رجیسترهای CSR مرتبط

این رجیستر CSR ویژهٔ دستور تأخیر سفارشی DLY طراحی شده و هستهٔ کنترل و وضعیتِ کارکرد تأخیر است. کارکردهای اصلی:

- **خواندن:** وضعیت جاری شمارندهٔ تأخیر (مانند مقدار شمارش و فلگ سرریز) را می‌خواند تا نرم‌افزار پیشرفت تأخیر را پرس‌وجو کند یا اتمام تأخیر را تشخیص دهد.
- **نوشتن:** پارامترهای تأخیر (مثل تعداد دورهای تأخیر و ضریب تقسیم ساعت) را پیکربندی می‌کند و پیکربندی لازم برای اجرای دستور DLY را فراهم می‌سازد.

رجیستر کنترل دستور تأخیر (U_NONS_DLY_0: 0x8C0):

جدول ۳-۲ رجیسترهای CSR همراهِ DLY

نام	بیت	کارکرد
dly_mask	31:24	ماسک پارامتر دستورِ تطبیق‌شده وقتی match=111 دستور dly است.
dly_data	23:16	مقدار {func، opcode} تطبیق‌شده وقتی match=111 دستور dly است.
CPU_dly_busy_sta	15	فلگ مشغول‌بودن هسته در وضعیت تأخیر.
dly_tout	14	اطلاعات اتمام‌وقت: در بازهٔ تأخیر هیچ درخواست دستورِ تطبیقی دریافت نشده است.
dly_ie	13	درخواست وقفهٔ تأخیر؛ پس از فعال‌سازی، اگر وضعیت اتمام‌وقت رخ دهد یک درخواست وقفهٔ نرم‌افزاری تولید می‌شود.
dly_priv	12	1: هم حالت ماشین و هم حالت کاربر مجاز به تطبیق‌اند؛ 0: عملیات delay اجراشده در حالت کاربر، دستورهای حالت ماشین را تطبیق نمی‌دهد.
Reserved	11:10	محفوظ.
dly_freq	9:0	پیکربندی ضریب تقسیم فرکانس شمارش تأخیر؛ اگر با فرکانس ساعت اصلی پیکربندی شود، تأخیر با واحد میکروثانیه تحقق می‌یابد.

۳.۲ ویژگی‌ها

۳.۲.۱ پیکربندی منبع تأخیر

- **انتخاب میان دو منبع ساعت:** با بیت div شمارش با ساعت اصلی یا ساعت تقسیم‌شده انتخاب می‌شود.
- **محاسبهٔ پویای تأخیر:** با بیت sel انتخاب می‌شود که مقدار تأخیر rs1 + imm (عدد ثابت) باشد یا rs1 + rs2 (مقدار رجیستر)؛ در نتیجه طول تأخیر در زمان اجرا به‌صورت پویا قابل تنظیم است.

- **پیکربندی تقسیم فرکانس:** با فیلد dly_freq در رجیستر U_NONS_DLY_0 ضریب تقسیم پیکربندی می‌شود؛ حتی می‌توان تأخیر را با واحد میکروثانیه (و ریزتر از آن) تنظیم کرد.

۳.۲.۲ کنترل خط لوله

با فیلد match این دستور تعیین می‌کند در طول تأخیر «چه چیزی» مسدود شود:

- **مسدودسازی دقیق:** می‌توان فقط دستورهای Load، Store یا Load/Store را مسدود کرد.
- **حالت مسدودسازی کامل:** با match=110 همهٔ دستورها مسدود می‌شوند که معادل تعلیق کامل خط لوله است.
- **مسدودسازی هوشمند:** با match=111 و پیکربندی رجیستر CSR، دستور با func3 و opcode مشخص تطبیق می‌یابد و دستورهای کارکردی خاص رهگیری می‌شوند.

۳.۲.۳ سازوکار بازخورد

رجیستر U_NONS_DLY_0 پایش وضعیت را فراهم می‌کند:

- **فلگ وضعیت:** CPU_dly_busy_sta نشان می‌دهد هسته در وضعیت تأخیر است یا نه.
- **تشخیص اتمام‌وقت:** dly_tout ثبت می‌کند که آیا در طول تأخیر درخواست دستورِ غیرمنتظره‌ای رخ داده است (یعنی دستوری که نبودنش درست بود، رخ داده است).
- **سازوکار وقفه:** پس از فعال‌شدن dly_ie، در صورت وقوع اتمام‌وقت می‌توان وقفهٔ نرم‌افزاری را برای دیباگ یا مدیریت استثنا فعال کرد.

۳.۳ استفاده

نسخهٔ جاری زنجیرهٔ ابزار هنوز پشتیبانی مونتاژگر دستور تأخیر سفارشی DLY را در خود ندارد. برای آنکه کاربر بتواند در محیط توسعهٔ موجود از کارکرد این دستور استفاده کند، کتابخانهٔ رسمی نرم‌افزار (SDK) مجموعه‌ای از توابع درون‌خطی (Intrinsic) به‌عنوان رابط جایگزین ارائه می‌کند:

جدول ۳-۳ رابط‌های قابل فراخوانی در کتابخانهٔ رسمی فعلی

رابط تابع	شرح کارکرد
DelayIntrinsic()	ورودی اصلی کارکرد تأخیر؛ اجرای تأخیر دقیق با تعداد دور مشخص
__set_DLY(value)	پیکربندی رجیستر تعداد دورهای تأخیر (نوشتن مقدار شمارش تأخیر)
__get_DLY()	خواندن مقدار جاری شمارندهٔ تأخیر
__clear_DLY_OV()	پاک‌کردن فلگ وضعیت سرریز تأخیر (Overflow)

نمونهٔ ساده:

```
__asm("nop");

__set_DLY(0, 0, ENABLE, ENABLE, 10);

DelayIntrisic(DLY_SRC_HCLK, DLY_Match_Load_Store, 0, 10);

__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");
__asm("nop");

__asm("lw x0, 0x00(x0)");
```

کد ۲ نمونهٔ استفاده از توابع درون‌خطی DLY

۱. آغاز زمان‌سنجی: پس از فراخوانی DelayIntrinsic، ماژول DLY بلافاصله پایش و زمان‌سنجی دستورها را در بازهٔ مشخص آغاز می‌کند.

۲. نتیجهٔ تطبیق دستور:

- **تطبیق موفق** (در بازهٔ دورهای مشخص دستور Load/Store شناسایی شد): ماژول DLY بلافاصله اجرای CPU را معلق و وارد وضعیت تأخیر دقیق می‌شود. پس از رسیدن به تعداد دورهای تأخیرِ تنظیم‌شده، تعلیق به‌طور خودکار برداشته می‌شود و CPU از دستور بعد از دستورِ تطبیق‌شده ادامه می‌دهد.
- **شکست تطبیق** (در بازهٔ دورهای مشخص هیچ دستور Load/Store شناسایی نشد): وقفهٔ نرم‌افزاری فعال می‌شود و به نرم‌افزار اطلاع می‌دهد که دستور هدف در بازهٔ مورد انتظار گرفته نشده است؛ در این حالت وضعیت تعلیقِ تأخیر رخ نمی‌دهد و تابع سرویس وقفه باید پردازش متناظر را انجام دهد.

۳. کاربرد تأخیر با تعداد دور ثابت (تطبیق دستور با خودش): اگر سناریوی کاربرد به تأخیر ثابت دور ساعت نیاز دارد، می‌توان نوع تطبیق را روی DLY_Match_Delay تنظیم کرد تا دستور DLY خودش را تطبیق دهد. در این حالت نیازی به دستور هدف اضافی نیست؛ پس از فراخوانی، ماژول DLY مستقیماً CPU را معلق و پس از تأخیرِ تعداد دور مشخص به‌طور خودکار ادامه می‌دهد. به شکل زیر:

```
while (1)
{
/* Perform intrinsic delay operation */
DelayIntrisic(DLY_SRC_HCLK, DLY_Match_Delay, 0, 100);
/* Toggle GPIOA pin 1 output state */
GPIOA->OUTDR ^= GPIO_Pin_1;
}
```

پیوست

تاریخچهٔ نسخه‌ها و بیانیه

جدول تاریخچه تاریخچهٔ نسخه‌های سند اصلی AN03003

تاریخ	نسخه	شرح تغییرات
2026/08/06	V1.0	انتشار نسخهٔ اول

بیانیه

این راهنما صرفاً جهت مرجع ارائه شده است و نویسندهٔ سند اصلی می‌تواند بدون اطلاع قبلی و در هر زمان، تغییراتی در محتوای آن اعمال کند. ترجمهٔ فارسی این سند توسط تیم معمار الکترونیک صنعت تهیه شده است؛ برای اطمینان از به‌روزترین نسخه به https://wch.cn مراجعه کنید.

پیوست

واژگان کلیدی (واژه‌نامهٔ فنی)

اصطلاحات فنی سخت‌افزار مطابق سند اصلی به‌صورت انگلیسی حفظ شده‌اند. جدول زیر معادل‌ها و شرح کوتاه هر واژه را برای مرجع سریع جمع می‌کند.

جدول واژگان واژگان کلیدی به‌کاررفته در این جزوه

واژه	یعنی چه؟
XW	توسعهٔ فشردهٔ سفارشی WCH؛ ۸ دستور ۱۶ بیتی برای بهینه‌سازی عملیات بایت و نیم‌کلمه
c.lbu / c.lhu / c.sb / c.sh	دستورهای فشردهٔ خواندن/نوشتن بایت و نیم‌کلمه در توسعهٔ XW؛ از فضای کدگذاری فشردهٔ ممیز شناور دودویی استفاده می‌کنند
-march=rv32imacxw	پارامتر کامپایل؛ باید شناسهٔ توسعهٔ xw صراحتاً اضافه شود تا کامپایلر پشتیبانی XW را فعال کند
MCPY	دستور سخت‌افزاری کپی پیوستهٔ بایت‌ها از آدرس مبدأ به مقصد؛ جایگزین memcpy بدون الزام هم‌ترازی
ucycle (0xC00)	رجیستر شمارش دور ماشین RISC-V؛ برای شمارش دقیق دورها در سنجش عملکرد
DLY	دستور تأخیر دقیق با دقت تک‌دوری؛ حذف عدم‌قطعیت زمان‌بندیِ حلقه‌های نرم‌افزاری
match	فیلد تطبیق نوع دستور در DLY؛ تعیین می‌کند در طول تأخیر «چه چیزی» مسدود شود
U_NONS_DLY_0 (0x8C0)	رجیستر CSR کنترل تأخیر؛ ماسک تطبیق، وضعیت مشغول‌بودن، اطلاعات اتمام‌وقت و ضریب تقسیم فرکانس
div / sel	انتخاب شمارش با ساعت اصلی یا تقسیم‌شده / انتخاب منبع مقدار تأخیر (imm یا rs2)
Intrinsic	توابع درون‌خطی کتابخانه (DelayIntrinsic() و...) برای استفاده از DLY در نبود پشتیبانی مونتاژگر

واژه	یعنی چه؟
PMP	حفاظت حافظهٔ فیزیکی؛ نقض مجوز در اجرای MCPY استثنای غیرقابل بازیابی می‌دهد
Hardware Breakpoint	نقطهٔ شکست سخت‌افزاری؛ برخورد در طول MCPY استثنا می‌دهد



معمار الکترونیک صنعت

MEMAR ELECTRONIC SANAT (MES)

در مورد مطالب این جزوه و ترجمه، بدیهی است که از نقطه‌نظرات و پیشنهادهای شما عزیزان در این راستا خوشحال می‌شویم. همچنین اشکالات، کم‌وکاستی‌ها و پرسش‌های خود را در این مورد با ما در میان بگذارید تا در نسخه‌های بعدی لحاظ شوند.

با تشکر، حسن معماری

ترجمهٔ فارسی سند AN03003 V1.0 · نسخهٔ ۱٫۰ — تاریخ ساخت: ۱۴۰۵/۰۸/۱۹

راه‌های ارتباط با ما

گیت‌هاب	https://github.com/MEMAR-ELECTRONIC-SANAT
کانال تلگرام	https://t.me/Memar_Electronic
پیج اینستاگرام	https://www.instagram.com/Memar.Electronic
ایمیل	Memari.h@gmail.com